

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS:

Total Marks: 30

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

- (a) Analyze the possible reasons for lack of responsiveness in the existing layout.
- (b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
- (c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
- (d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
- (e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

- (a) Analyze the issues in the existing form validation mechanism.
- (b) Design input fields for email and phone number with appropriate HTML attributes.
- (c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
- (d) Develop a mechanism to display **real-time error messages** for invalid inputs.
- (e) Evaluate the validation system and suggest enhancements for better user experience and security.

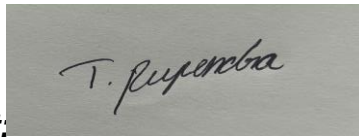
Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

- Analyze the CSS properties responsible for layout overlapping issues.
- Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
- Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
- Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
- Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

*I T RUPENDRA/(192311325)Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Responsive Layout Design

(a) Analyze the possible reasons for lack of responsiveness

A retail website may not be responsive due to the following reasons:

1. Fixed-width layouts using pixels (px) instead of flexible units (% , vw, rem).
2. No media queries to adjust the layout for different screen sizes.
3. Images and videos without responsive sizing.
4. Improper use of CSS positioning and margins.
5. Navigation menu not optimized for smaller screens.
6. Lack of Flexbox or CSS Grid for flexible alignment.

(b) Design a Responsive Webpage using CSS Flexbox

HTML (index.html)

```
<!DOCTYPE html><html><head>
  <title>Responsive Retail Website</title>
  <link rel="stylesheet" href="style.css"></head><body>
<header>
  <h1>Retail Store</h1></header>
<nav>
  <a href="#">Home</a>
  <a href="#">Products</a>
  <a href="#">Offers</a>
  <a href="#">Contact</a></nav>
<div class="container">
  <aside class="sidebar">
    <h3>Categories</h3>
    <ul>
      <li>Electronics</li>
      <li>Fashion</li>
      <li>Home Appliances</li>
    </ul>
  </aside>

  <main class="content">
    <h2>Welcome to Our Store</h2>
    <p>
      Shop the latest products with attractive discounts.
      This webpage automatically adjusts according to screen size.
    </p>
```

```
    </main></div>
</footer>
    <p>© 2026 Retail Store. All Rights Reserved.</p></footer>
</body></html>
```

CSS (style.css)

```
*{
    margin:0;
    padding:0;
    box-sizing:border-box;
    font-family:Arial, sans-serif;
}
body{
    background:#f4f4f4;
}
header{
    background:#0066cc;
    color:white;
    text-align:center;
    padding:20px;
}
nav{
    display:flex;
    justify-content:center;
    gap:20px;
    background:#333;
    padding:15px;
}
nav a{
    color:white;
    text-decoration:none;
}

.container{
    display:flex;
    gap:20px;
    padding:20px;
}

.sidebar{
    flex:1;
    background:white;
    padding:20px;
}

.content{
    flex:3;
    background:white;
    padding:20px;
}
```

```

footer{
    background:#222;
    color:white;
    text-align:center;
    padding:15px;
    margin-top:20px;
}

```

(c) Media Queries for Mobile, Tablet, and Desktop

```

/* Mobile */@media (max-width:600px) {
nav{
    flex-direction:column;
    align-items:center;
}

.container{
    flex-direction:column;
}

}
/* Tablet */@media (min-width:601px) and (max-width:992px) {

.container{
    flex-direction:column;
}
nav{
    justify-content:space-around;
}

}
/* Desktop */@media (min-width:993px) {

.container{
    flex-direction:row;
}

}

```

(d) Spacing, Alignment, and Distribution Techniques

- Used **Flexbox** for flexible layout.
- Added `gap` to maintain equal spacing.
- Applied `padding` for proper internal spacing.
- Used `justify-content` and `align-items` for alignment.
- Applied `box-sizing: border-box` for consistent sizing.
- Maintained equal spacing between navigation items and content sections.

(e) Evaluation and Further Improvements

Effectiveness

- Responsive on mobile, tablet, and desktop devices.
- Proper alignment of header, navigation, content, and footer.
- Improved readability with balanced spacing.
- Flexible layout reduces content overlap and misalignment.
- Better user experience across different screen sizes.

Further Improvements

1. Use **CSS Grid** for more complex page layouts.
2. Add a hamburger menu for mobile navigation.
3. Optimize images using responsive image techniques.
4. Improve accessibility with semantic HTML and ARIA labels.
5. Enhance performance using lazy loading and CSS optimization.

Expected Output

- **Desktop:** Sidebar and content displayed side by side.
- **Tablet:** Sidebar appears above the content.
- **Mobile:** Navigation stacks vertically, and all sections are displayed in a single column with proper spacing and alignment.

Experiment 2: Form Validation using JavaScript

(a) Analyze the issues in the existing form validation mechanism

The existing registration form may have the following issues:

1. No validation for email and phone number formats.
2. Users can submit empty or invalid inputs.
3. No real-time feedback while entering data.
4. Incorrect data gets stored in the database.
5. Poor user experience due to late error detection.
6. Increased chances of invalid or duplicate registrations.

(b) Design Input Fields with HTML Attributes

HTML (index.html)

```
<!DOCTYPE html><html><head>
```

```

<title>Registration Form Validation</title>
<link rel="stylesheet" href="style.css"></head><body>
<h2>User Registration Form</h2>
<form id="myForm">
<label>Email:</label><br><input type="email" id="email" placeholder="Enter
Email" required><span id="emailError"></span>
<br><br>
<label>Phone Number:</label><br><input type="text" id="phone"
placeholder="Enter 10-digit Number"maxlength="10" required><span
id="phoneError"></span>
<br><br>
<input type="submit" value="Register">
</form>
<script src="script.js"></script>
</body></html>

```

(c) Implement JavaScript Validation using Regular Expressions

JavaScript (script.js)

```

const email = document.getElementById("email");const phone =
document.getElementById("phone");
const emailError = document.getElementById("emailError");const phoneError =
document.getElementById("phoneError");
email.addEventListener("keyup",
validateEmail);phone.addEventListener("keyup", validatePhone);
function validateEmail() {
let pattern=/^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if(pattern.test(email.value)) {
    emailError.innerHTML="Valid Email";
    emailError.style.color="green";
}else{
    emailError.innerHTML="Invalid Email";
    emailError.style.color="red";
}
}
function validatePhone() {
let pattern=/^[0-9]{10}$/;
if(pattern.test(phone.value)) {
    phoneError.innerHTML="Valid Phone Number";
    phoneError.style.color="green";
}else{
    phoneError.innerHTML="Enter a valid 10-digit number";
    phoneError.style.color="red";
}
}

```

```
document.getElementById("myForm").addEventListener("submit",function(e){
let emailPattern=/^[^\s@]+@[^\s@]+\.[^\s@]+$/;let phonePattern=/^[0-9]{10}$/;
if(!emailPattern.test(email.value) || !phonePattern.test(phone.value)){
    alert("Please enter valid details.");
    e.preventDefault();
}
});
```

(d) Real-Time Error Messages

The validation system displays instant feedback while the user types.

- **Valid Email:** "Valid Email" (Green)
- **Invalid Email:** "Invalid Email" (Red)
- **Valid Phone Number:** "Valid Phone Number" (Green)
- **Invalid Phone Number:** "Enter a valid 10-digit number" (Red)

(e) Evaluation and Further Improvements

Effectiveness

- Prevents invalid email and phone number submission.
- Provides real-time validation and instant feedback.
- Improves data accuracy.
- Enhances user experience.
- Reduces invalid database entries.

Further Improvements

1. Validate all required fields (Name, Password, Address, etc.).
2. Add password strength validation.
3. Display validation icons along with messages.
4. Perform server-side validation for security.
5. Protect against SQL Injection and Cross-Site Scripting (XSS).
6. Use HTTPS and CAPTCHA to improve security.

Expected Output

- User enters a **valid email** (e.g., `user@example.com`) → **"Valid Email"** displayed in green.
- User enters an **invalid email** (e.g., `user@`) → **"Invalid Email"** displayed in red.
- User enters a **valid 10-digit phone number** (e.g., `9876543210`) → **"Valid Phone Number"** displayed in green.
- User enters an **invalid phone number** (e.g., `12345`) → **"Enter a valid 10-digit number"** displayed in red.

- On clicking **Register**, the form is submitted only if both fields are valid; otherwise, an alert is shown and submission is prevented.

Experiment 3: CSS Layout Debugging

(a) Analyze the CSS properties responsible for layout overlapping issues

Layout overlapping may occur due to the following reasons:

1. Excessive use of `position: absolute`.
2. Fixed widths and heights that do not adapt to different screen sizes.
3. Missing responsive layouts using Flexbox or CSS Grid.
4. Improper margins and padding causing element collisions.
5. Missing media queries for different screen resolutions.
6. Incorrect use of `z-index`, resulting in elements appearing on top of each other.

(b) Examine the Role of the CSS Box Model

The CSS Box Model determines how the size and spacing of an element are calculated.

Box Model Component	Description
Content	The actual text or image inside the element.
Padding	Space between the content and the border.
Border	Surrounds the padding and content.
Margin	Space outside the border that separates elements.

Importance:

- Prevents elements from touching each other.
- Maintains proper spacing and alignment.
- Improves readability and overall layout consistency.
- Helps create responsive web pages.

(c) Debug the Layout Using Appropriate Positioning Techniques

HTML (index.html)

```
<!DOCTYPE html><html><head>  
  <title>CSS Layout Debugging</title>  
  <link rel="stylesheet" href="style.css"></head><body>  
<header>Website Header</header>  
<div class="container">
```

```
<div class="box">Section 1</div><div class="box">Section 2</div><div  
class="box">Section 3</div>  
</div>  
<footer>Website Footer</footer>  
</body></html>
```

CSS (style.css)

```
*{  
    margin:0;  
    padding:0;  
    box-sizing:border-box;  
}  
body{  
    font-family:Arial;  
}  
header, footer{  
    background:#0077cc;  
    color:white;  
    text-align:center;  
    padding:20px;  
}  
  
.container{  
    display:flex;  
    flex-wrap:wrap;  
    gap:20px;  
    justify-content:center;  
    padding:20px;  
}  
  
.box{  
    flex:1 1 250px;  
    background:#f4f4f4;  
    padding:30px;  
    text-align:center;  
    border:1px solid #ccc;  
}
```

Positioning Techniques Used

- `display: flex` for flexible layouts.
- `flex-wrap: wrap` prevents overlapping.
- `box-sizing: border-box` ensures correct sizing.
- Relative positioning can be used when slight adjustments are required without removing elements from the normal document flow.

(d) Modify the CSS to Prevent Overlapping During Resizing

Media Queries

```
/* Mobile */@media (max-width:600px) {  
  
  .container{  
    flex-direction:column;  
  }  
  
  .box{  
    width:100%;  
  }  
  
}  
/* Tablet */@media (min-width:601px) and (max-width:992px) {  
  
  .box{  
    flex:1 1 45%;  
  }  
  
}  
/* Desktop */@media (min-width:993px) {  
  
  .box{  
    flex:1 1 30%;  
  }  
  
}
```

Changes Made

- Used Flexbox instead of absolute positioning.
- Enabled wrapping with `flex-wrap`.
- Added responsive media queries.
- Used flexible widths instead of fixed pixel values.
- Applied consistent spacing with `gap`, `margin`, and `padding`.

(e) Evaluation and Best Practices

Evaluation

- Layout adapts correctly to mobile, tablet, and desktop screens.
- No overlapping of elements during browser resizing.
- Improved spacing and alignment.
- Better readability and user experience.
- Responsive design ensures consistent display across devices.

Best Practices

1. Use **Flexbox** or **CSS Grid** for responsive layouts.
2. Avoid excessive use of `position: absolute`.
3. Use relative units such as `%`, `rem`, `vw`, and `vh` instead of fixed pixels.

4. Apply `box-sizing: border-box` to all elements.
5. Use media queries to support different screen sizes.
6. Maintain consistent spacing with `margin`, `padding`, and `gap`.
7. Test layouts on multiple browsers and devices.

Expected Output

- **Desktop:** Three content sections are displayed side by side with equal spacing.
- **Tablet:** Two sections appear in the first row and one in the second row without overlapping.
- **Mobile:** All sections are stacked vertically with proper alignment and spacing.
- The header and footer remain properly aligned, and the layout stays responsive without overlapping during browser resizing.